

人工神经网络期末报告

模型结构

整体结构

本次实验中使用的模型是一个基于Transformer的Seq2Seq模型。它由Encoder, Decoder和Projection Layer组成。它的输入是源语言token序列和目标语言输入序列，输出是目标语言token的概率分布。

Encoder

模型的Encoder由1个Embedding层、1个PositionalEncoding层、2个Encoder层、1个归一化层组成。Encoder的输入是源语言token序列和源序列的padding mask，输出是每个token的上下文语义表示。其中，Embedding层是pytorch提供的，用于将源语言token id映射为稠密向量。PositionalEncoding层用于添加token的位置信息。每个Encoder层包括1个MultiHeadAttention层、1个FeedForward层，2个归一化层和1个dropout层。

Decoder

模型的Decoder由1个Embedding层、1个PositionalEncoding层、2个Decoder层、1个归一化层组成。Decoder的输入是目标语言token序列和源序列的padding mask，输出是目标语言每个token的上下文语义表示。其中，Embedding层用于将目标语言token id映射为d_model维的向量，PositionalEncoding层用于添加token的位置信息。每个Decoder层包括2个MultiHeadAttention层、1个FeedForward层，3个归一化层和1个dropout层。

Projection Layer

模型的线性映射层的功能是将Decoder的输出特征映射到目标词表大小的维度上，计算每个位置生成各个词的概率分布。

代码补全过程

在代码补全过程中，完成了这些模块的实现。

Positional Encoding

Positional Encoding是将位置信息注入token的嵌入表示的方法。

Positional Encoding共有两个步骤：在初始化的时候构建位置编码矩阵，在前向传播的时候给输入加上位置编码。

构建位置编码矩阵的公式如下：

$$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$
$$\text{PE}_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

在构建位置编码矩阵时，首先需要构造除数项，随后计算奇数位置（cos）和偶数位置（sin）的值。最后需要扩展位置编码矩阵的维度，从而在前向传播中自动广播。

在前向传播中，输入token嵌入，给token嵌入加上位置编码，告诉模型这个单词在哪里，并返回处理后的token嵌入。

MultiHeadAttention

多头注意力层需要三个线性层作为注意力头，分别关注Query, Key, Value。最后还需要一个线性层聚合注意力，一个Dropout层缓解过拟合。

在前向传播中，首先需要将维度转换为多头形式，随后计算点积注意力，并用softmax得到注意力分布，最后使用dropout缓解过拟合。在得到注意力分布前，如果模型使用了掩码，还需要在注意力得分中应用掩码，从而遮挡掉不应该被模型看到的部分。

EncoderLayer

EncoderLayer的作用是生成源语言token的上下文语义表示。在前向传播中，首先需要通过多头注意力层提取源语言上下文关系，同时利用残差连接和层归一化稳定训练。随后，通过一个FFN，增强非线性建模能力，并再次使用残差连接和层归一化稳定训练。

DecoderLayer

EncoderLayer的作用是生成目标语言token的上下文语义表示。在前向传播中，首先需要通过多头注意力层提取目标语言上下文关系。随后，通过一个交叉注意力层，让目标语言的每个位置可以融合源语言上下文信息。最后，使用一个前馈神经网络增强非线性建模能力。在每一个步骤中，都利用残差连接和层归一化稳定训练。

实验结果分析

```
Building prefix dict from the default dictionary ...
Loading model from cache /tmp/jieba.cache
Loading model cost 0.556 seconds.
Prefix dict has been built successfully.
```

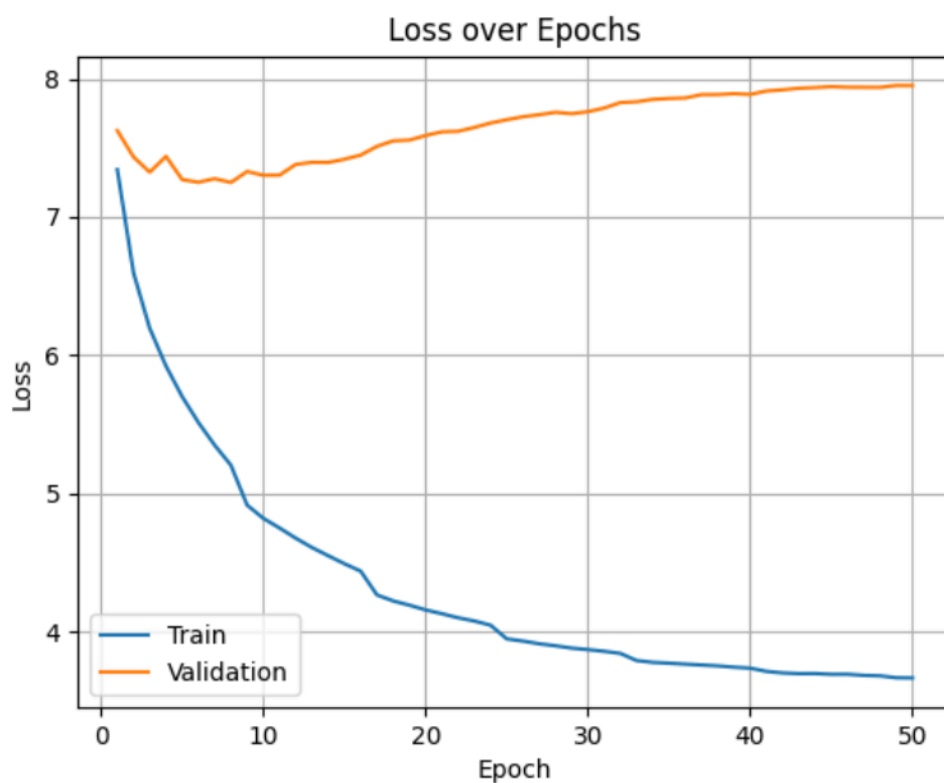
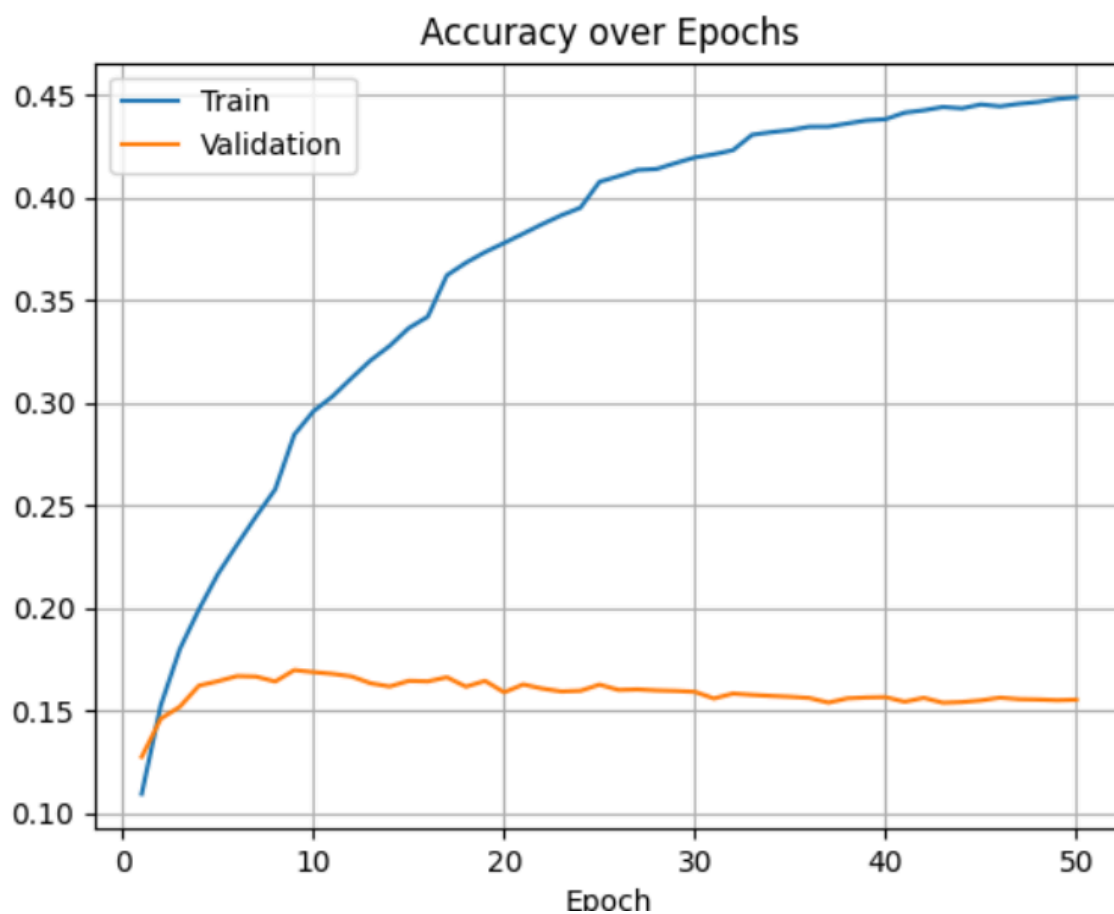
```
Corpus BLEU: 0.60
Translations saved to translations.json
```

BLEU分数是自然语言处理中用来评价机器翻译质量的指标。测试BLEU分数为0.6，说明翻译可以理解，流畅性较好。

```
▼ 2:
src: "“听起来你被锁住了啊，”<unk>回复道。"
ref: "<unk> like you are <unk> the Deputy <unk> <unk>"
hyp: "A few years ago were called for the more than a few years ago that the people to the more than the more than the more than the
y were two decades of the long time."
hyp_id: "0bcdb15b2526386bd9110dff68599fad676f4474362079956c92c3ebc05cde03"
▼ 3:
src: "白宫将此次“美国制造”活动定义为官方活动，因此不受《<unk>法案》管辖。"
ref: "The <unk> in <unk> event was designated an official event by the White House, and would not have been covered by the <unk> Ac
t."
hyp: "The US will be a recent US will not have a official American official development of official development of official developm
ent of official development of official development assistance."
hyp_id: "579dde02b8c2e64f5b51561d88b3d4621e8892f42fef34429e57c692cd7fe3b1"
► 4:
. . .
```

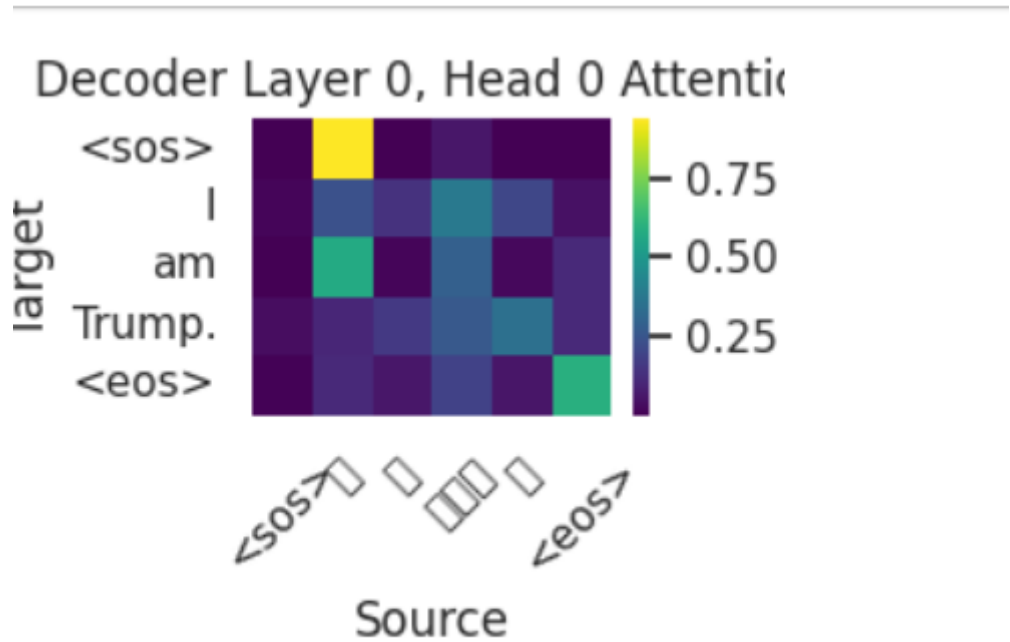
在translations.json中可以看到原文、参考人类译文和机器译文。

可视化分析



随着训练进行，训练集损失下降、准确率上升，测试集损失仅在前几个epoch下降、后续开始上升，准确率也前几个epoch上升、后续下降，说明出现了过拟合，可能与数据集规模过小有关。

我尝试过使用100k训练集，但GPU显存有限，将batch_size调整到4依然无法正常训练，所以放弃了进一步提升准确率。



为了模型增强可解释性，我对Transformer中的注意力权重进行了可视化，绘制了模型的热力图。这里的源语言句子是“我是特朗普。”，翻译目标是“I am Trump.”。从图中可以看出，模型的注意力分布有问题。生成 <sos> 时，模型的注意力主要集中在中文“我”字上。生成“I”时，注意力主要集中在中文“特朗普”上。生成“am”时，注意力在“我”上。生成“Trump”时，注意力在“。”上。这也与译文中出现的问题相对应。

个人总结

通过本次实验，我掌握了Seq2Seq Transformer模型的结构，对Transformer架构、Encoder、Decoder有了更深的理解。除此之外，我掌握了如何生成热力图、如何从热力图中分析模型在训练中的注意力强度分布，从而了解模型的训练过程。